Guía de Actividades **Práctico-Experimentales** Nro. 002

1. Datos Generales

Asignatura	Estructura de Datos
Ciclo	3ro A / B
Unidad	Estructuras de datos estáticas (arreglos, conjuntos, registros, archivos) y estructuras de datos dinámicas (listas, pilas y colas)
Resultado de aprendizaje de la unidad	R1. Implementa algoritmos simples de búsqueda y explica las diferencias en el orden de complejidad computacional, bajo los principios de solidaridad,
Práctica Nro.	02
Título de la Práctica	Implementación de acceso secuencial y aleatorio a archivos con métricas de I/O
Nombre del Docente	Cristian Ramiro Narváez Guillén
Fecha	jueves 8 de octubre 2026 - viernes 9 de octubre 2026.
Horario	jueves 8 de 08:30 – 09:30 viernes 9 de 11:30 – 13h30
Lugar	EVA Aula
Tiempo planificado en el Sílabo	3 horas
Nombres de los estudiantes	Eduardo José soto Cabrera Christopher Alexander Pineda Rodas Lenin Fabricio Macas Cabrera Willan Sebastian Granda Camacho Jeancarlos Fernando Aguirre Romero

2. Objetivo(s) de la Práctica:

Implementar algoritmos de acceso secuencial para la persistencia masiva de datos en disco mediante el módulo nativo de I/O en JavaScript.

Cuantificar empíricamente la latencia computacional (tiempo de respuesta $T(n)$) y el impacto en la memoria Heap ($S(n)$) al procesar archivos de forma síncrona (bloqueante) versus flujos asíncronos (*Streams*).

Evaluar las decisiones de diseño físico en el acceso a almacenamiento mecánico o de estado sólido bajo los principios de *Green Computing*, analizando cómo la optimización de los accesos I/O reduce el consumo energético del hardware.

3. Materiales y reactivos:

- Software Node.js (versión LTS 20.x o superior) instalado en el sistema.
- IDE moderno para desarrollo (Visual Studio Code, WebStorm).
- Conexión a Internet estable para consultas a la documentación técnica oficial de Node.js (módulo fs).
- Acceso a la consola del sistema (Terminal de Linux/macOS o PowerShell en Windows).
- Recursos Digitales: Sílabo de la asignatura, repositorio base en GitHub.
- Plataforma EVA UNL.

4. Equipos y herramientas

Recursos Físicos

- Aula de Computación.

Computadora personal o de laboratorio:

- Sistema operativo: Windows, Linux o macOS.
- Procesador: Mínimo 2 núcleos (recomendado 4).
- Memoria RAM: Mínimo 4 GB.
- Almacenamiento: Disco HDD o SSD con al menos 1 GB de espacio libre temporal.

5. Procedimiento / Metodología.

Enfoque PID-2025:

- **ABP (Aprendizaje Basado en Problemas):** El equipo satelital del APE 001 ahora necesita leer el millón de coordenadas ($N = 10^6$) que han sido transmitidas y almacenadas en un archivo físico en el servidor. El problema radica en que leer el archivo de una sola vez colapsa la memoria RAM del contenedor en la nube. Se debe diseñar una estrategia para leer los datos con el mínimo impacto temporal y espacial.
- **ABI (Aprendizaje Basado en Investigación):** El estudiante investigará cómo interactúa el *Event Loop* de JavaScript (motor V8 y libuv) con las llamadas del sistema operativo (Syscalls) para el acceso a disco.

- **Modalidad:** Individual y análisis comparativo grupal posterior.

Tarea 1: Generación del Conjunto de Datos Masivo (Mock Data)

Para poder medir el I/O, primero necesitamos un archivo físico considerable.

1. En el directorio de su proyecto, cree un archivo llamado lab02_generador.js.
2. Implemente el siguiente código para escribir un archivo CSV con 10^6 registros utilizando escritura en flujo (Stream) para no saturar la memoria durante su creación.

```
const fs = require('fs');
const { performance } = require('perf_hooks');

const FILE_NAME = 'coordenadas_masivas.csv';
const N = 1000000;

console.log(`[Generador] Iniciando escritura de ${N} registros...`);
const start = performance.now();

// Utilizamos un Stream de escritura (Write Stream)
const writableStream = fs.createWriteStream(FILE_NAME);
writableStream.write('id,latitud,longitud\n'); // Cabecera

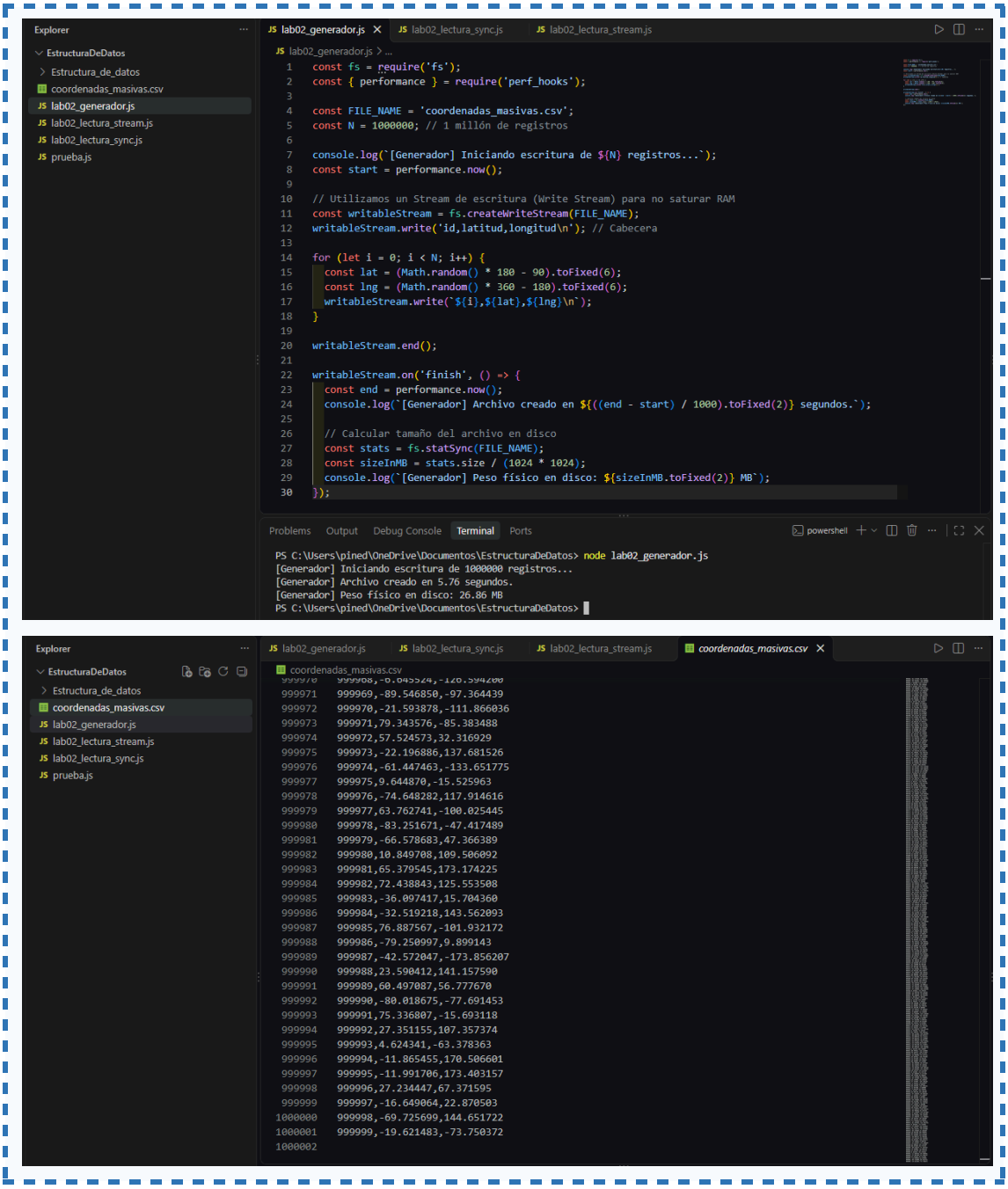
for (let i = 0; i < N; i++) {
  const lat = (Math.random() * 180 - 90).toFixed(6);
  const lng = (Math.random() * 360 - 180).toFixed(6);
  // Buffer string format
  writableStream.write(`${i},${lat},${lng}\n`);
}

writableStream.end();

writableStream.on('finish', () => {
  const end = performance.now();
  console.log(`[Generador] Archivo creado en ${((end - start) /
1000).toFixed(2)} segundos.`);
    // Calcular tamaño del archivo en disco
```

```
const stats = fs.statSync(FILE_NAME);
const sizeInMB = stats.size / (1024 * 1024);
console.log(`[Generador] Peso físico en disco: ${sizeInMB.toFixed(2)} MB`);
});
```

3. Ejecute en terminal: node lab02_generador.js y verifique la creación del archivo coordenadas_masivas.csv.



Tarea 2: Enfoque 1 - Lectura Síncrona (Bloqueante / $O(n)$ Espacial)

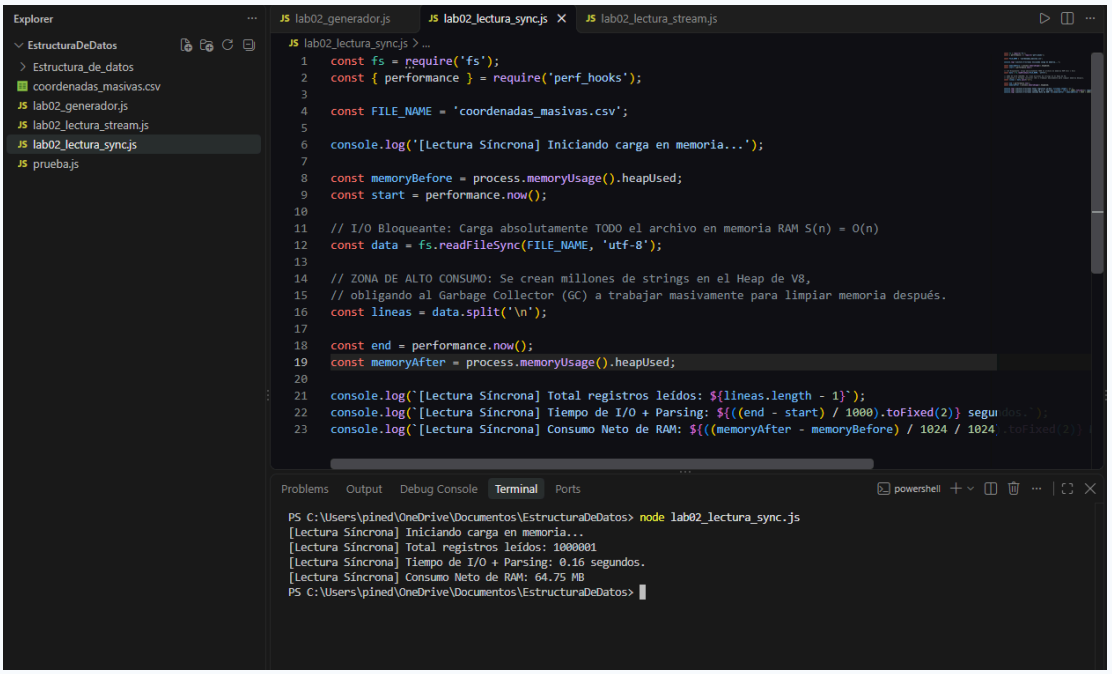
Este enfoque representa la manera más intuitiva, pero menos eficiente, de procesar un archivo físico.

1. Cree un archivo llamado lab02_lectura_sync.js.
2. Implemente el código para cargar todo el archivo en la RAM.

```
const fs = require('fs');
const { performance } = require('perf_hooks');
const FILE_NAME = 'coordenadas_masivas.csv';
console.log(`[Lectura Síncrona] Iniciando carga en memoria...`);
const memoryBefore = process.memoryUsage().heapUsed;
const start = performance.now();
// I/O Bloqueante: El hilo principal se detiene hasta cargar todo
const data = fs.readFileSync(FILE_NAME, 'utf-8');
// Simulamos procesamiento separando por saltos de línea
const lineas = data.split('\n');
const end = performance.now();
const memoryAfter = process.memoryUsage().heapUsed;
console.log(`[Lectura Síncrona] Total registros leídos: ${lineas.length - 1}`);
console.log(`[Lectura Síncrona] Tiempo de I/O + Parsing: ${((end - start) / 1000).toFixed(2)} segundos.`);
console.log(`[Lectura Síncrona] Consumo Neto de RAM: ${((memoryAfter - memoryBefore) / 1024 / 1024).toFixed(2)} MB`);
```

3. Ejecute node lab02_lectura_sync.js y tabule el tiempo y el impacto en memoria.

1859



Tarea 3: Enfoque 2 - Lectura Asíncrona con Streams ($O(1)$ Espacial)

1. Cree el archivo lab02_lectura_stream.js.
2. Implemente la abstracción de flujos (Streams) para leer fragmentos (*chunks*) secuenciales, reduciendo la complejidad espacial teórica de $S(n) = n \cdot \text{bytes}$ a un constante $S(n) = O(1)$ correspondiente al tamaño del *buffer*.

```
const fs = require('fs');
const { performance } = require('perf_hooks');
const FILE_NAME = 'coordenadas_masivas.csv';
console.log(` [Lectura por Stream] Iniciando procesamiento... ` );
const memoryBefore = process.memoryUsage().heapUsed;
const start = performance.now();
let totalRegistros = 0;
// Creamos un flujo de lectura secuencial no bloqueante
const readableStream = fs.createReadStream(FILE_NAME, { encoding: 'utf-8'
});
readableStream.on('data', (chunk) => {
   // Procesamos el chunk de datos actual sin almacenarlo completo
   // Contamos cuántos saltos de línea hay en este fragmento
```

```
    let lineBreakCount = (chunk.match(/\n/g) || []).length;
    totalRegistros += lineBreakCount;
});
readableStream.on('end', () => {
    const end = performance.now();
    const memoryAfter = process.memoryUsage().heapUsed;
    console.log(`[Lectura por Stream] Total registros procesados:
${totalRegistros}`);
    console.log(`[Lectura por Stream] Tiempo de I/O parcializado: ${((end -
start) / 1000).toFixed(2)} segundos.`);
    console.log(`[Lectura por Stream] Consumo Neto de RAM: ${((memoryAfter
- memoryBefore) / 1024 / 1024).toFixed(2)} MB`);
});
```

3. Ejecute node lab02_lectura_stream.js. Compare drásticamente la métrica del consumo neto de RAM entre la Tarea 2 y la Tarea 3.

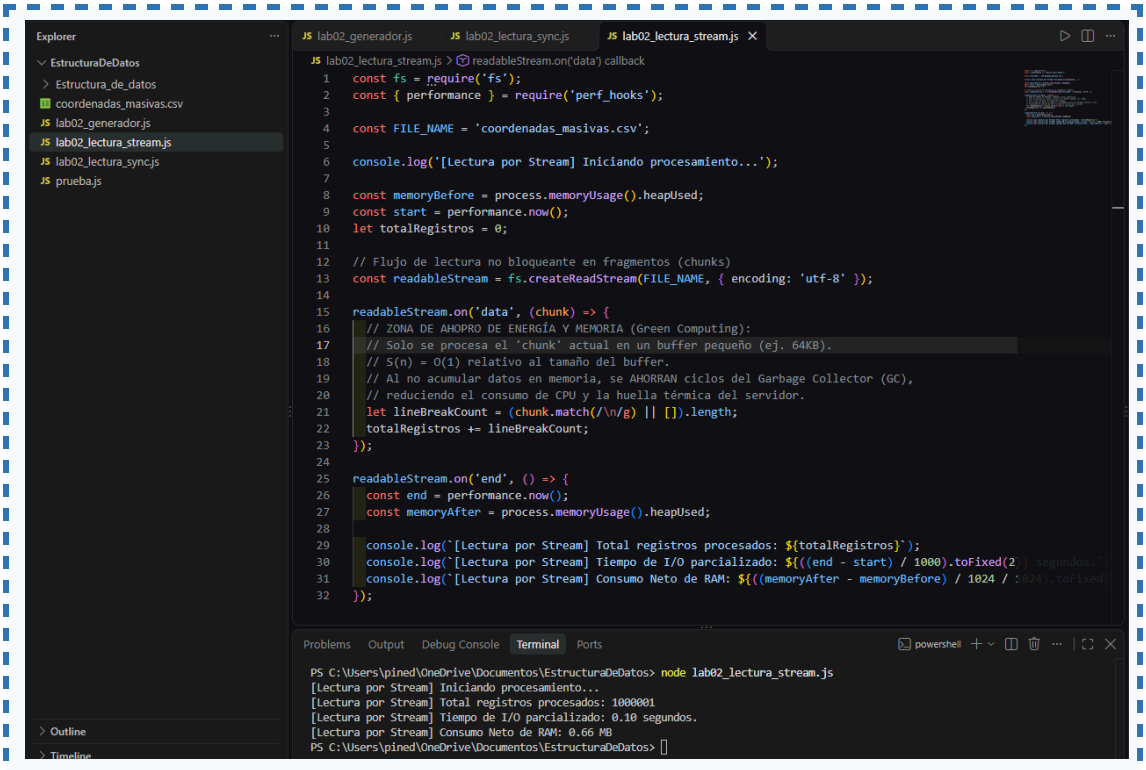


Tabla comparativa (complete con sus mediciones de las Tareas 2 y 3):

Métrica	Lectura síncrona (medido)	Lectura por Stream (medido)
Registros leídos	1000001	1000001
Tiempo (segundos)	0.19 s	0.09 s

Métrica	Lectura síncrona (medido)	Lectura por Stream (medido)
Consumo neto de RAM (MB)	64.75 MB	0.66 MB

Tarea 4: Análisis Matemático de Rendimiento (ABI)

Basado en los resultados empíricos, los estudiantes investigarán el diseño del bus de datos y el *Thread Pool* interno (libuv) que gestiona estas llamadas.

- Demuestre teóricamente (utilizando notación de orden asintótico) por qué la función bloqueante impone $S(n) = O(n)$ frente al enfoque *Stream* $S(n) = O(1)$.
- Estructure su respuesta relacionándola con el uso energético del servidor (menos RAM implica menor recolección de basura y menor gasto térmico del procesador).

Resolución de la Tarea 4:

4.1 Investigación (ABI): Event Loop, libuv y llamadas al sistema

Node.js combina V8 (ejecuta JavaScript y administra el *heap* con su recolector de basura), libuv (*Event Loop*, I/O asíncrono y *Thread Pool*) y enlaces en C++ hacia el sistema operativo. Los archivos regulares no pueden gestionarse con sondeo no bloqueante (epoll/kqueue los consideran siempre "listos"), por lo que libuv envía las llamadas bloqueantes (open, read, fstat, close) a un hilo del *Thread Pool* (4 por defecto, ajustable con UV_THREADPOOL_SIZE).

Flujo de fs.createReadStream: (1) JavaScript solicita una lectura; (2) libuv encola la petición en el *Thread Pool*; (3) un hilo trabajador ejecuta read(2) y el kernel, con el controlador del disco y DMA, copia el bloque a RAM; (4) al terminar, el resultado vuelve al *Event Loop* y su fase de *poll* ejecuta el *callback* en el hilo principal; (5) el flujo emite el evento "data" con un *chunk* de highWaterMark = 64 KiB y solicita el siguiente, hasta el evento "end". En cambio, readFileSync ejecuta la syscall en el propio hilo principal, **detiene el Event Loop** y reserva un *Buffer* del tamaño total del archivo. Los *streams* no reducen los bytes leídos; reducen el **pico de memoria** y evitan bloquear el hilo principal.

4.2 Demostración asintótica

Definiciones. n = número de registros; b = bytes promedio por registro (≈ 28,5 B por el formato id,latitud,longitud); F = n·b = tamaño del archivo; K = highWaterMark (64 KiB), constante respecto de n; S(n) = pico de memoria viva durante la ejecución.

Lectura bloqueante. (1) readFileSync(..., "utf-8") debe entregar el contenido completo: lee F bytes a un *Buffer* de tamaño F y los decodifica a una cadena de longitud ≈ F, así que el pico es ≥ F. (2) data.split("\n") crea un arreglo de n + 2 elementos, cada uno con un objeto cadena de tamaño constante h (cabecera del objeto y referencia). (3) Sumando: $S_{sync}(n) \geq F + n \cdot h = n \cdot (b + h) = \mathbf{\Omega(n)}$. Además, todo se reserva una vez y es proporcional a F, de modo que $S_{sync}(n) \leq c \cdot n$ para alguna constante c. Por lo tanto $\mathbf{S_{sync}(n) = \Theta(n) = O(n)}$: cualquier algoritmo que cargue el archivo completo necesita al menos F bytes.

Lectura por Stream. Invariante: en cualquier instante el flujo retiene a lo sumo (a) un *chunk* de tamaño ≤ K (en el buffer de lectura y su cadena decodificada), (b) el arreglo temporal que devuelve match(), de a lo sumo K elementos, y (c) el contador totalRegistros (un número). Cada *chunk* queda inalcanzable al terminar el *callback*. Entonces $S_{stream}(n) \leq c_1 \cdot K + c_2 \cdot K + c_3 = O(K)$, y como K no depende de n, $\mathbf{S_{stream}(n) = O(1)}$. Lo que sí crece con n es el número de *chunks*, $\lceil F/K \rceil = \Theta(n)$, y con él el tiempo, no el espacio.

Comparación. Ambos enfoques tienen $T(n) = \Theta(n)$ (hay que leer y examinar los F bytes), pero $S_{sync}(n) = \Theta(n)$ frente a $S_{stream}(n) = \Theta(1)$, y por tanto $\lim S_{stream}(n) / S_{sync}(n) = 0$ cuando $n \to \infty$. Para $n = 10^6$ la relación de buffers es $F/K \approx 28{,}5 \cdot 10^6 / 65\,536 \approx 435$, es decir, del orden de 400 veces menos memoria. Con $n = 10^9$ (≈ 28 GB) la versión síncrona ni siquiera es viable (fallaría con errores como ERR_FS_FILE_TOO_LARGE o ERR_STRING_TOO_LONG), mientras que el *stream* sigue usando unos 64 KiB.

4.3 Relación con el uso energético del servidor

La energía consumida es E = P·t, y la potencia dinámica de la CPU crece con la actividad ($P \approx C \cdot V^2 \cdot f$). Menos ciclos de CPU y menos memoria activa implican menos energía por tarea:

- **Garbage Collector.** Con readFileSync el *heap* llega a contener del orden de n objetos de larga vida; V8 debe promoverlos a la generación antigua y los recolecta con ciclos mayores (Mark-Sweep-Compact), cuyo costo es proporcional a los objetos vivos, O(n) por ciclo, además de pausar el hilo principal. Con *streams*, los *chunks* nacen y mueren jóvenes y se limpian con recolecciones menores (*Scavenge*), baratas

porque casi no hay objetos vivos que copiar. Menos trabajo del GC significa menos ciclos de CPU y menos gasto térmico.

- **Memoria RAM.** La DRAM consume energía para refrescar las celdas ocupadas. Una huella plana permite contenedores más pequeños, mayor densidad de servicios por servidor y evita el *swap*, que añade I/O adicional. Si un servidor atiende 100 solicitudes simultáneas sobre este archivo, la versión síncrona necesitaría ≈ 100 × 27 MB ≈ 2,7 GB, y la de *streams* ≈ 100 × 64 KiB ≈ 6,25 MiB.
- **Almacenamiento.** Ambos enfoques leen los mismos bytes, por lo que el ahorro no proviene de leer menos sino de hacerlo de forma secuencial y en bloques: en un HDD se evitan desplazamientos mecánicos del cabezal (milisegundos y energía) y en un SSD se aprovecha la lectura anticipada de la caché de páginas.
- **Salvedad honesta:** con 27 MB la diferencia energética absoluta es pequeña; el beneficio se vuelve significativo al escalar n y el número de solicitudes concurrentes.

4.4 Conclusión

Con el mismo costo temporal asintótico, el enfoque por *streams* reduce la complejidad espacial de $\Theta(n)$ a $\Theta(1)$, mantiene libre el hilo principal, disminuye la presión sobre el *Garbage Collector* y la RAM, y por ello resulta más escalable y sostenible bajo los principios de *Green Computing*.

6. Resultados esperados:

- Se obtendrán métricas directas que evidencian cómo una operación aparentemente trivial de lectura/escritura puede causar cuellos de botella de I/O (*Input/Output Bottleneck*). El uso de *Streams* demostrará que la persistencia masiva puede gestionarse secuencialmente manteniendo una huella de memoria plana, logrando un código ambiental y computacionalmente sostenible.

Resolución del análisis de resultados esperados:

Qué se espera observar. Ambos enfoques leen exactamente los mismos bytes del mismo archivo (≈ 27 MB, 10^6 registros más la cabecera), por lo que el costo temporal es del mismo orden, $T(n) = \Theta(n)$, típicamente décimas de segundo en un equipo actual. La diferencia aparece en la **memoria**: la lectura síncrona mantiene todo el contenido y las líneas parseadas en el *heap* ($S(n) = \Theta(n)$), mientras que el *stream* solo retiene un *chunk* de 64 KiB a la vez ($S(n) = \Theta(1)$ respecto de n). Esto evidencia el cuello de botella de I/O y de memoria que causa una operación aparentemente trivial como leer un archivo.

Métrica	Tarea 2: readFileSync (referencia esperada)	Tarea 3: Streams (referencia esperada)
Tamaño del archivo	≈ 27 MB (mismo archivo)	≈ 27 MB (mismo archivo)
Registros contados	1 000 001 (1 000 000 + cabecera)	1 000 001 (cuenta los "\n", incluida la cabecera)
Tiempo total	Décimas de segundo (≈ 0,2–0,8 s)	Mismo orden (≈ 0,1–0,5 s); no es necesariamente menor
Consumo neto de RAM (heapUsed)	Decenas a ~100 MB (≈ 70–120 MB), crece con n	Pocos MB o menos; puede ser ≈ 0 o incluso negativo por el GC
Hilo principal	Bloqueado durante toda la lectura	Libre: la lectura la hace libuv en segundo plano
Complejidad espacial	S(n) = Θ(n)	S(n) = Θ(1) (tamaño del buffer)

Valores de referencia estimados; las mediciones reales dependen del hardware, de la versión de Node.js y del momento en que actúe el *Garbage Collector*.

Observaciones técnicas para interpretar las capturas:

- El conteo de registros sale 1 000 001 en ambos scripts porque incluye la línea de cabecera (id,latitud,longitud); no es un error.
- heapUsed no incluye la memoria externa del *Buffer* que crea readFileSync (≈ 27 MB adicionales), así que el costo real del enfoque síncrono es **mayor** que el reportado. Si desea medirlo completo, imprima también process.memoryUsage().rss y .external.
- Un consumo neto cercano a 0, o negativo, en el *stream* es normal: el GC pudo liberar memoria previa durante la lectura. Lo relevante es que **no crece con n**.
- En el generador (Tarea 1) el ciclo for es síncrono y no respeta la contrapresión (*backpressure*): write() devuelve false cuando el buffer interno se llena, pero el ciclo sigue escribiendo, por lo que gran parte del contenido queda encolado en memoria antes de vaciarse a disco. Para memoria realmente constante se debe esperar el evento "drain" cuando write() devuelva false.

Conclusión: la persistencia masiva puede gestionarse de forma secuencial con una huella de memoria plana, lo que da un código más sostenible (menos presión sobre el GC, menos RAM reservada y menos energía por tarea) sin perder correctitud.

7. Preguntas de Control:

- Desde una perspectiva filosófica profunda y analítica, responda:

1. **Causa Material:** Al realizar una lectura síncrona mediante readFileSync, ¿cuál es la "materia" que se satura en el ordenador y cómo se diferencia físicamente del estado de reposo de un archivo almacenado magnética o electrónicamente en el disco duro?

   Respuesta:

   La materia que se satura es la **memoria principal (RAM)**, concretamente el *heap* de V8 y su memoria externa asociada. Con readFileSync los F bytes del archivo deben estar a la vez en acto en ese sustrato: primero un *Buffer* del tamaño del archivo, luego la cadena decodificada y, tras split, un arreglo con un millón de cadenas. La RAM (DRAM) guarda cada bit como carga eléctrica en un condensador que debe **refrescarse constantemente**; mientras más celdas se ocupan, mayor es la presión sobre el *Garbage Collector* y, si se excede el límite del contenedor, aparece el uso de *swap* o el error de memoria agotada.

   En reposo, el archivo es materia **en potencia**: en un HDD son dominios magnéticos orientados en el plato; en un SSD, carga atrapada en celdas de memoria NAND. Es un estado estable, **no volátil**, que se conserva sin gastar energía y que la CPU no puede operar directamente. En RAM esa misma información pasa a ser materia **en acto**: volátil, sostenida por energía, accesible en nanosegundos y escasa. La lectura no “vacía” el disco; copia un patrón de estados (la forma) a otro sustrato.

2. **Causa Formal:** El archivo CSV (Valores Separados por Comas) posee una "forma" específica (texto secuencial lineal). ¿De qué manera esta forma impone restricciones algorítmicas sobre nuestra capacidad para acceder aleatoriamente a la coordenada $N = 500,000$ sin tener que recorrer las $499,999$ coordenadas previas?

   Respuesta:

   El CSV es texto lineal con registros de **longitud variable** separados por “\n” (por ejemplo “7,-12.3,4.5” frente a “499999,-89.123456,-179.654321”) y sin tabla de posiciones. La posición en bytes de la línea k es la suma de las longitudes de las k−1 líneas anteriores, un valor que no se conoce sin haberlas recorrido. Para llegar al registro N = 500 000 hay que leer y contar saltos de línea desde el inicio (aprox. 14 MB de los ≈ 28 MB del archivo), es decir, un costo **$\Theta(k)$** y no O(1).

Se puede recuperar el acceso aleatorio **agregando estructura** a esa forma: (a) registros de ancho fijo, donde la posición se calcula como cabecera + k·R y se salta con un desplazamiento (fs.read con el parámetro position, o createReadStream con la opción start); (b) un **índice auxiliar** id → desplazamiento, construido en una pasada O(n) y consultado luego en O(1) u O(log n); o (c) si los id están ordenados, una **búsqueda binaria por bytes** (saltar a la mitad y resincronizar en el siguiente salto de línea). En todos los casos hay que conocer o construir el desplazamiento; la forma "texto secuencial" por sí sola solo garantiza acceso secuencial.

3. **Causa Eficiente:** Durante el uso de *Streams* asíncronos en Node.js, el *Event Loop* delega la operación al sistema operativo. En este contexto, ¿quién actúa como el verdadero motor (causa eficiente) que transfiere los electrones desde el disco hasta la memoria RAM en fragmentos (chunks)?

   Respuesta:

   Ni el *Event Loop* ni el código JavaScript mueven datos: el *Event Loop* es la causa **coordinadora** que despacha la petición y, al terminar, ejecuta el *callback* en el hilo principal. La cadena causal real es: (1) el módulo *fs* de Node crea una petición hacia libuv; (2) un hilo del *Thread Pool* de libuv (4 por defecto, configurable con UV_THREADPOOL_SIZE) ejecuta la llamada bloqueante al sistema read(2); (3) el **kernel del sistema operativo** (sistema de archivos virtual, caché de páginas, planificador de bloques y controlador del dispositivo) atiende esa llamada; y (4) el **controlador del disco** (SATA/NVMe), mediante **DMA** (acceso directo a memoria), deposita los bloques leídos en la RAM sin que la CPU copie byte a byte.

   Por tanto, el verdadero motor (causa eficiente en sentido propio) es el **sistema operativo junto con el hardware de almacenamiento y su controlador DMA**; el hilo de libuv es la causa próxima o instrumental y el *Event Loop* solo orquesta y notifica. Lo que viaja no son "electrones del disco" como tales, sino un patrón de estados (magnéticos en un HDD, de carga en un SSD) que se transcribe a señales eléctricas y se re-codifica en las celdas de la RAM.

4. **Causa Final:** Si todo dato en ejecución debe residir en la memoria principal para ser computado por la CPU, ¿cuál es el propósito (telos) de la "persistencia" en el disco físico, sabiendo que las operaciones de

I/O añaden un costo de tiempo monumental en la escala de microsegundos (μs) frente a los nanosegundos (ns) de la memoria caché?

Respuesta:

El telos de la persistencia es que el dato **siga siendo** más allá del proceso que lo computa. La RAM es rápida pero volátil, pequeña y cara por gigabyte; el disco es lento pero no volátil, grande y barato. La persistencia cumple tres fines: (1) **durabilidad**, porque el dato sobrevive al cierre del programa, a fallos y al apagado; (2) **capacidad**, porque permite conservar más información de la que cabe en memoria principal; y (3) **compartición y recuperación**, porque distintos procesos o máquinas pueden acceder al mismo dato y reconstruir el estado tras una caída.

La latencia es el precio de esa permanencia, y por eso existe la jerarquía de memoria. Órdenes de magnitud aproximados: caché L1 ≈ 1 ns; RAM ≈ 100 ns; SSD NVMe ≈ 10–100 µs; HDD ≈ 5–10 ms (miles de veces más lento que la RAM). La CPU solo computa sobre lo que está en memoria principal, pero el disco es la "memoria a largo plazo" del sistema. De ahí que el buen diseño busque **minimizar y ordenar el I/O**: acceso secuencial, lectura en bloques, *buffering*, cachés y *streams*, que es justamente lo que mide esta práctica.

5. **Esencia y Existencia:** Un conjunto de 10^6 coordenadas existe como un bloque monolítico en el archivo data.csv, pero en el enfoque de *Streams*, solo una porción existe en la RAM a la vez. ¿Es conceptualmente el "mismo" conjunto de datos procesado, si en la memoria jamás llegó a existir de manera unificada? Argumente.

Respuesta:

Sí: es el mismo conjunto de datos; lo que cambia es su **modo de existencia**, no su esencia. En términos aristotélicos, la identidad de las 10^6 coordenadas la determina su **forma** (qué valores contiene, cuántos registros son y en qué orden están), no la simultaneidad espacial de su **materia**. El archivo en disco es el conjunto existiendo como unidad (en potencia, listo para ser computado); el *stream* lo hace pasar al acto por partes: cada *chunk* existe en RAM un instante y luego desaparece, pero todos provienen del mismo sustrato y recorren exactamente la misma secuencia de bytes. Es como leer un libro página a página:

nunca tiene todas las páginas ante los ojos a la vez, y aun así se lee el mismo libro.

La condición es que el resultado buscado sea **descomponible**: contar registros, sumar, hallar máximos o mínimos pueden obtenerse combinando resultados parciales de cada fragmento (un acumulador de tamaño O(1)), y por eso el procesamiento por flujo equivale al procesamiento unificado. En cambio, operaciones con visión global (ordenar todo el conjunto, calcular una mediana exacta, eliminar duplicados) exigen estado adicional o algoritmos externos, porque ahí la "unidad" del conjunto debe reconstruirse en algún lugar. También hay que cuidar el borde entre fragmentos: un *chunk* puede cortar un registro por la mitad, y preservar la identidad del dato implica reensamblar ese registro (el conteo de saltos de línea del laboratorio no lo sufre, pero un parseo línea por línea sí).

8. Evaluación

Criterio Evaluado	Puntaje Máximo	Evidencia Esperada
1. Generación de Data	2 puntos	Captura de consola que demuestre la creación exitosa del archivo de ~ y su peso impreso.
2. Implementación de I/O Bloqueante	2 puntos	Captura de consola ejecutando lab02_lectura_sync.js, mostrando el alto impacto en el consumo de RAM.
3. Implementación de I/O por Streams	2 puntos	Captura ejecutando lab02_lectura_stream.js evidenciando el

		consumo neto de RAM cercano a 0 o muy bajo.
4. Respuestas Aristotélicas y ABI	3 puntos	Informe en PDF detallando las respuestas de control y el análisis matemático $S(n)$ vs $O(1)$.
5. Sustentabilidad Computacional	1 punto	Código comentado indicando la zona exacta donde se ahorran ciclos del Garbage Collector.
Total	10 puntos	

9. Bibliografía

- P. W. Bible and L. Moser, *An Open Guide to Data Structures and Algorithms*. PALNI Open Press, 2023. [Online]. Available: https://open.palni.org/
- Node.js Foundation, "File system | Node.js v20.x Documentation", *Node.js API*, 2023. [Online]. Available: https://nodejs.org/docs/latest-v20.x/api/fs.html
- S. Tilkov et al., *Node.js in Action*, 2nd ed. Manning Publications, 2017.

10. Elaboración y Aprobación

- Elaborado por: nombre del Docente responsable de la práctica.
- Aprobado por: nombre del Director de Carrera.

Elaborado por	Cristian R. Narváez Guillén **Docente**	
Aprobado por	Edison L Coronel Romero **Director de Carrera**	
